

Choosing a stack when AI writes the code

Decision order, the options at each layer, and a ranked shortlist of full stacks

The premise. Enterprise web software (HRIS, finance, inventory) plus mobile, built by directing AI rather than hand-coding. Application-level performance is not a constraint. The consequence: the bottleneck is no longer *authoring* code, it is **verifying** it and containing the damage when verification fails. Every criterion below follows from that single shift.

Why the order matters more than it used to

AI made the *last* decision the cheapest to reverse and left the first ones exactly as expensive as they always were. Regenerating a service in a different framework used to be a quarter of work; now it is closer to a week. But no amount of model capability makes it cheap to change your primary-key strategy after two years of production data, to unpick a distributed architecture, or to retrofit sync onto a schema that never anticipated it.

So spend judgment in inverse proportion to how much airtime each decision usually gets. Stack selection attracts 90% of the debate and is perhaps 20% of the outcome.

#	Layer	Cost to reverse
1	Data model + database	Brutal
2	Sync posture	Very high — and it constrains layer 1
3	Architecture	High
4	Client surface shape	Moderate
5	Stack	Real, but now the cheapest of the five

The five decisions, in order

1. Data model and database

Options: Postgres · SQL Server · MySQL/MariaDB · Oracle

Default: Postgres. SQL Server is the one legitimate alternative — on .NET and shipping on-prem, many buyers already hold licences and have DBAs. MySQL gives up too much for financial workloads. Oracle is a cost decision disguised as a technical one.

The decisions *inside* this layer matter more than which engine:

- **Multi-tenancy.** Shared schema + *tenant_id* + row-level security (default) · schema-per-tenant · database-per-tenant where buyers demand hard isolation. Nearly impossible to change later.
- **Financial data is append-only.** A posted journal entry is never mutated; corrections are reversal entries. If AI can UPDATE a posted ledger row, you have no audit trail whatever the audit log says.
- **Keys.** UUIDv7, not autoincrement, so clients can generate IDs offline.
- **Temporal from day one.** HRIS and finance are *as-of* domains. Effective dating and retroactive adjustment belong in the model, not bolted on later.

2. Sync posture

Decide this *with* the data model, because it dictates the schema.

Options: server-authoritative outbox queue with idempotency keys (thin client, no local reads) · full bidirectional sync engine (PowerSync, ElectricSQL, Couchbase Lite; local store via Drift/SQLite)

Default: Outbox queue for HRIS and finance — they do not actually need offline reads. A real sync engine for the warehouse only. Buy it; do not let AI improvise one.

If you choose bidirectional anywhere, the schema needs *updated_at* and version columns on everything syncable, tombstones instead of hard deletes, and a written conflict rule per table.

3. Architecture

Options: modular monolith · monolith plus extracted operational services · microservices · serverless

Default: One deployable per product, modular monolith inside, boundaries enforced by architecture tests (NetArchTest, ArchUnit, import-linter). Extract only under demonstrated operational pressure: batch, reporting, sync, webhook ingestion, document rendering.

API contract: OpenAPI plus generated clients is the default — machine-checkable across a seam that now spans app versions you do not control. gRPC for internal service-to-service. tRPC only if TypeScript sits both ends. GraphQL no: it buys client flexibility you do not need and costs authorization complexity you cannot verify.

4. Client surface shape

Options: PWA only · PWA plus one cross-platform app · cross-platform everywhere · native per platform

Default: PWA for HRIS and finance self-service; one Flutter app with server-driven screens where you need push, biometrics or store presence; native only if a scanner SDK forces it.

5. Stack

Everything above is settled first. The table below ranks only this last layer.

The ranked shortlist

V	Machine-checkable correctness — how much wrongness is caught with no human in the loop
C	Corpus size and coherence — AI proficiency, tooling maturity, hireable reviewers
B	Boring — low churn, so AI is not writing yesterday's deprecated patterns
1	Context economy — total conceptual surface needed to make one change
Bat	Enterprise batteries off the shelf — SSO, RBAC, audit, migrations, jobs, i18n, reporting
Ops	Resilience to recurring dependency and build maintenance (5 = low tax)

Postgres assumed throughout, so it is not scored. **Weighted** counts V, Bat and Ops double, reflecting the two concerns that actually bind you: verification burden and recurring maintenance tax.

#	Full stack	V	C	B	1	Bat	Ops	Raw	Weighted
1	.NET + Blazor + Flutter	5	4	4	3	5	4	25	39
—	<i>PWA only, no native app</i>	4	5	3	5	3	5	25	37 *
2	Django + HTMX/React + Flutter	3	5	4	3	5	4	24	36
2=	Laravel + Livewire + Flutter	3	5	4	3	5	4	24	36
4	Go/sqlc + TS web + Flutter	5	4	5	2	2	5	23	35
4=	.NET + Blazor + MAUI	5	3	3	5	4	3	23	35 **
6	Kotlin/Spring + Compose Multiplatform	5	3	3	4	4	3	22	34
7	TypeScript end-to-end + Flutter	4	4	3	3	3	4	21	32
8	TypeScript end-to-end + React Native/Expo	4	5	2	3	3	2	19	28
9	Dart everywhere (Serverpod + Flutter)	5	2	2	5	2	2	18	27

* Cannot cover warehouse scanning — read as “use wherever sufficient”, not as a contender. Note that it scores second-highest, which is the point.

** The row where the columns deserve more trust than the total. Its Ops 3 is generous: a thin maintainer community is the plugin-tax mechanism we are trying to avoid. Score Ops 2 and it is 33.

What the reweighting changed

.NET holds first, and by more. Weighting all three real concerns pulls it further ahead, because it is the only stack scoring 4 or better on every one of them.

Go rose to fourth and stopped there. Verifiability and operational resilience are best-in-class — one toolchain, sub-second builds, the strongest compatibility promise in the industry, compile-checked SQL via sqlc. Two points of missing batteries, doubled, is what holds it. That is the honest arithmetic of owning everything yourself.

Django and Laravel are more robust than their verifiability suggests. Doubling batteries rewards them for code that does not exist to be verified. That is not a scoring artefact; it is the real effect. Run pyright strict or PHPStan level 9 and treat it as non-negotiable.

Flutter is assumed as the mobile layer in almost every row. The plugin-maintenance tax is not an authoring problem, so AI does not solve it: forking an abandoned plugin converts a dependency into new native code in the languages where both AI and your reviewers are weakest, and Gradle-CocoaPods-npm version resolution is close to AI's worst case. Flutter's first-party package set, single resolver and owned rendering pipeline attack exactly that.

Nothing rescues React Native under weights that take operational tax seriously.

The caveat that matters

The columns are not independent. Batteries is a leading indicator of Verifiable — every missing battery becomes bespoke, unverified code. Ops and Common are correlated too, through maintainer-community size. The totals therefore compress genuinely different risks into one number and should not be read as precision.

Their real use is diagnostic: find the column you are most afraid of, and see who is weak in it.

Recommendation .NET + Blazor + Flutter + Postgres. Modular monolith, one deployable per product, OpenAPI codegen across every seam, PWA wherever it is sufficient, server-driven screens on mobile.

Then stop theorising

Build one genuinely hard vertical slice — the **offline barcode scan-and-sync**, not the leave-request form, which will work in anything. Measure iterations to green tests, bugs that escaped to manual QA, how legible the output is to a reviewer who did not commission it, and most diagnostically, how well a *fresh* AI session modifies it a week later.

Alongside it, check the dependency graph: take the two or three plugins you would actually depend on — hardware scanner integration, secure storage, background location — and look at maintainer activity, issue response times, and whether they are first-party.

Stack choice is roughly 20% of the outcome here. Spec discipline, module boundaries and review process are the other 80%.